

RepoMind AI: A Multi-Agent System for Intelligent Repository Analysis and Task Automation

Submitted By

Anshuman Sarkar

Roll No. : 012/CSC/241002

Registration No. : 012-1111-0943-21

Akash Chakraborty

Roll No. : 012/CSC/241001

Registration No. : 611-1111-0296-21

Supervisor

Mrs. Shilpa Saha

Faculty Member, Department Of Computer Science, Asutosh College

A Final Project Report submitted to the Department of Computer Science, Asutosh College, University of Calcutta in partial fulfilment of the requirements for the Degree of

M.Sc. IN COMPUTER SCIENCE

2024-2026

Certificate

This is to certify that the project (Paper Code: **CSMP 403**) topic entitled “**Repo-Mind AI: A Multi-Agent System for Intelligent Repository Analysis and Task Automation**” has been solely prepared and submitted by Anshuman Sarkar (Roll No: 012/CSC/241002, Reg. No: 012-1111-0943-21) and Akash Chakraborty (Roll No: 012/CSC/241001, Reg. No: 611-1111-0296-21), under the supervision of Mrs. Shilpa Saha, Faculty Member, Department of Computer Science, Asutosh College, Kolkata during the year 2025 in partial fulfilment of Computer Science Project examination conducted by University of Calcutta.

(Supervisor)

Mrs. Shilpa Saha
Faculty Member
Department of Computer Science,
Asutosh College, Kolkata, India.

Head of The Department

Department of Computer Science,
Asutosh College, Kolkata, India.

Examiner(s)

Acknowledgement

We would like to express our heartfelt gratitude and deep appreciation to all those who have supported and guided us throughout the course of this project.

First and foremost, we would like to extend our sincere thanks to Prof. Anika Sinha, Head of the Department and our project supervisor, Department of Computer Science, Asutosh College, University of Calcutta, Kolkata, for giving us the opportunity to do the research-oriented project work and providing invaluable guidance throughout this project study. Her dynamism, vision, sincerity, and motivation have deeply inspired us. She has taught us the methodology to carry out the research and to present the project work as clearly as possible.

We also thank the authority of Asutosh College and every member of the Department of Computer Science, Asutosh College, University of Calcutta, Kolkata, for their support in carrying out this work.

Last but not least, we express our deepest gratitude to our family and friends for their unwavering support, encouragement, and understanding during this journey.

This project would not have been possible without the contribution and support of all the above individuals.

Anshuman Sarkar

Akash Chakraborty

INDEX

CHAPTER NO.	SL. NO.	STATEMENT	PAGE NO.
	–	Abstract	1–2
1	1	Project Overview	3
	2	Project Objective	4
	3	Why Choose RepoMind AI	4–5
	4	Previous Projects and Their Evaluation Criteria	5–6
	5	Feature Comparison	7
	6	Gaps We Are Trying to Solve	7–8
	7	Problem Statement	8
	8	Positioning of RepoMind AI	8
2	1	Repository Input	9
	2	File Scanning and Filtering	9
	3	Code Chunking	9
	4	Embedding and Indexing	9
	5	Repo-Aware Chat	9
	6	Repository Navigation	10
	7	Architecture Understanding	10
	8	Bug Investigation	10
	9	Change Planning	10
	10	Approval-Based Editing	10
	11	Review and Testing Guidance	10
	12	Documentation Generation	11
	13	Commit and PR Assistance	11
	14	Overall Workflow	11
	15	Methodology Summary	12
	16	Overall Workflow of the Project	12
3	1	Repository Input	13
	2	High-Level Technology Stack	13–14
	3	System Components and Responsibility	14–15
	4	Comprehensive Agent Implementation and Algorithm	15–28
	5	Step-by-Step Technical Flows	28–31
	6	Result and Discussion	31–39
	7	Architectural Highlight	40–41
	8	Key Presentation Takeaways	41
4	1	System Transparency and Data Flow	42–45
	2	Automated Test Suite Breakdown	45

CHAPTER NO.	SL. NO.	STATEMENT	PAGE NO.
	3	The Result and Execution Log	45
	4	Assessments of Testing Gaps	46
	5	Manual Verification Flow	46
5	—	Comparison Table	47–49
6	1	Limitations	50–51
	2	Future Scope	51–52
	3	Conclusion	52

Abstract

Understanding the architecture of a software project is often one of the most difficult tasks for developers. Large repositories contain many files, folders, modules, and dependencies, making it difficult to understand how different parts of the system work together. Developers, especially beginners and new contributors, spend a significant amount of time exploring project structures, locating important files, understanding code flow, finding bugs, and creating documentation. Existing AI coding tools mainly focus on code generation and editing, but they provide limited support for architecture understanding and repository analysis.

RepoMind AI is a repository-aware AI assistant designed to help developers understand, analyze, and improve software repositories more efficiently. The system allows users to import repositories from GitHub or upload project files. It then scans and filters project files, divides code and documentation into smaller chunks, creates embeddings, and stores repository knowledge in a vector database for fast retrieval.

The main feature of RepoMind AI is its **Architecture Analysis Engine**. The system automatically studies the repository structure and identifies important components such as frontend, backend, database, services, testing modules, configuration files, and documentation. It generates architecture summaries, file-structure views, dependency relationships, and system diagrams, helping developers quickly understand how the project is organized and how different components interact with each other.

The proposed system is built using **Python, FastAPI, React.js, LangChain, ChromaDB, GitHub API, and Large Language Models (LLMs)**. It uses **Retrieval-Augmented Generation (RAG)** to retrieve repository information before generating responses. This allows the system to provide accurate and context-aware answers based on the actual repository content.

In addition to architecture analysis, RepoMind AI provides repository-aware chat, intelligent code search, project summarization, bug investigation, documentation generation, change planning, code review, testing guidance, approval-based editing, and commit or pull request support. Before making any modification, the system creates a structured plan and requires user approval, ensuring safe and controlled development.

The main objective of RepoMind AI is to simplify repository understanding and software maintenance by combining architecture analysis, repository indexing, semantic search, and AI-powered assistance into a single platform. By providing a clear view of project architecture and intelligent repository guidance, the system helps developers learn, maintain, and improve software projects more effectively. The proposed system aims to reduce learning time, improve developer productivity, and make software development easier and more efficient.

Keywords: Architecture Analysis, Repository-Aware AI, Retrieval-Augmented Generation

(RAG), Large Language Models (LLMs), Software Repository Analysis, Semantic Search, Code Understanding, Documentation Generation, Bug Investigation, Vector Database, GitHub Integration.

Chapter 1

Introduction

1 Project Overview

RepoMind AI is a repository-aware AI assistant designed to help developers understand, analyze, and manage software repositories more efficiently. Modern software projects often contain hundreds of files, making it difficult for developers to understand project structure, locate important code, investigate bugs, and maintain documentation. RepoMind AI addresses these challenges by providing intelligent repository analysis and AI-assisted development support.

The system allows users to import a repository from GitHub and automatically scans, filters, and indexes the project files. Using embeddings, vector databases, and Retrieval-Augmented Generation (RAG), RepoMind AI builds an understanding of the repository and retrieves relevant code sections when answering user queries.[5]

A key feature of RepoMind AI is its Architecture Analysis Engine, which studies the repository structure and identifies important components such as frontend, backend, database, services, testing modules, configuration files, and documentation. It generates architecture summaries and system diagrams to help developers quickly understand how the project works.[7]

In addition to architecture analysis, RepoMind AI provides repository-aware chat, semantic code search, project summarization, bug investigation, documentation generation, change planning, code review, testing guidance, and commit message generation. The system follows an approval-based workflow where all code modifications require user confirmation before being applied, ensuring safe and controlled development.

Built using Python, FastAPI, React.js, LangChain, ChromaDB, GitHub API, and Large Language Models (LLMs), RepoMind AI combines repository understanding, semantic search, and AI-powered assistance into a single platform. The project aims to reduce onboarding time, improve developer productivity, simplify repository maintenance, and provide a safer and more transparent software development experience.[9]

2 Project Objective

The primary objective of RepoMind AI is to design and implement a secure, codebase-aware multi-agent development assistant that automates repository-level tasks such as planning, code editing, bug investigation, regression review, and commit generation while maintaining the structural integrity of the codebase.

To achieve this objective, the system focuses on three core technical goals:

1. **Context-Aware Automation (RAG):**

Bridge developer intent and repository knowledge through dense vector indexing, semantic search, and efficient evidence retrieval. This enables the system to provide accurate, repository-aware answers and recommendations.

2. **Deterministic-First Reliability:**

Apply strict static validation rules, including syntax checking, bracket verification, and file structure validation, before using Large Language Models (LLMs). This improves reliability, reduces errors, and minimizes API computation costs.

3. **Human-in-the-Loop Security:**

Enforce user approval for all critical file modifications through controlled workflows. The system provides diff previews, rollback support, and safety checks to prevent unintended changes and code regressions.

3 Why Choose RepoMind AI?

Even with many AI coding tools available today, developers still face several challenges while working with repositories: [13]

- Understanding old or unfamiliar projects takes a lot of time.
- Large folder structures are difficult to navigate.
- Finding specific files, functions, or bugs is not easy.
- Project architecture and diagrams are hard to create manually.
- Existing tools provide limited guidance for understanding how a project works.
- Getting a short summary of a large codebase is difficult.
- Adding new features often lacks proper planning and impact analysis.
- Code quality, testing, and review support are limited.
- Most tools do not explain repository content in the user's preferred language.

- Developers often need multiple tools for documentation, planning, review, and code assistance.

RepoMind AI solves these problems by providing repository understanding, architecture explanation, code search, project summarization, bug investigation, planning, documentation generation, code review, testing guidance, multilingual support, and approval-based editing in a single platform.

4 Previous Projects and Their Evaluation Criteria

Several AI-powered software development tools have been developed to improve coding productivity, repository understanding, and software maintenance. These systems provide features such as code generation, repository-aware chat, code search, and automated editing. However, each system focuses on specific aspects of software development and still leaves gaps.

4.1 Continue (github.com/continuedev/continue)

Description: Continue provides open-source AI workflows across IDE, terminal, and CI environments. It supports chat, editing, autocomplete, and custom agents. [6]

Evaluation Criteria:

- Open-source AI workflows
- IDE and terminal integration
- Chat and autocomplete support
- Custom agent support

4.2 Tabby (github.com/TabbyML/tabby)

Description: Tabby is a self-hosted AI coding assistant and an alternative to GitHub Copilot. [6]

Evaluation Criteria:

- Self-hosted code assistant
- Local model deployment
- Code completion and suggestions

4.3 SWE-bench

Description: SWE-bench evaluates whether AI models can solve real GitHub issues using repository context. [2]

Evaluation Criteria:

- Repository-level problem solving
- Real-world GitHub issue dataset
- Code understanding capability

4.4 OpenHands

Description: OpenHands provides autonomous software development agents for task execution. [15]

Evaluation Criteria:

- Autonomous task execution
- Code interaction and modification
- AI development agents

4.5 GitHub Copilot

Description: GitHub Copilot is an AI coding assistant that helps generate and complete code. [6]

Evaluation Criteria:

- Code completion
- Code generation
- IDE-based assistance

4.6 Cursor

Description: Cursor is an AI-powered code editor with repository-aware chat and editing features. [6]

Evaluation Criteria:

- Codebase-aware chat
- Multi-file editing
- AI-assisted development

5 Feature Comparison

Feature	Continue / Tabby	SWE-bench / OpenHands	GitHub Copi- lot / Cursor	RepoMind AI
Repo-aware Chat	Yes	Partial	Partial	Yes
Architecture Ex- planation	No	No	Partial	Yes
Bug Investigation	Partial	Yes	No	Yes
Documentation Generation	No	No	Partial	Yes
Approval-Based Editing	No	Partial	No	Yes
Learning-Oriented Focus	No	No	No	Yes
RAG-Based An- swers	Partial	No	No	Yes
Project Summa- rization	Partial	No	Partial	Yes
Code Search	Yes	Partial	Partial	Yes
Change Planning	No	Partial	No	Yes

Table : Feature Comparison [4]

6 Gaps We Are Trying to Solve

1. Slow Repository Understanding

Developers spend a lot of time reading files to understand how a project works and how it is organized. [1]

2. Difficult Error Finding and Debugging

Finding bugs and identifying the cause of errors in large projects can be time-consuming and challenging. [9]

3. Lack of Automation

Many development tasks such as project analysis, documentation, planning, and code review still require manual effort. [13]

4. Unsafe Automated Editing

AI-generated code changes can be risky if they are applied without user approval,

change preview, or rollback support. [5]

5. Weak Documentation Support

Many projects do not have proper README files, architecture summaries, setup guides, or developer documentation. [7]

7 Problem Statement

Modern software projects often contain a large number of files, folders, modules, and dependencies. When developers work on a new or unfamiliar repository, they spend a significant amount of time understanding the project structure, locating important files, finding features, and identifying bugs.

Although tools like GitHub Copilot, Cursor, and others help with code generation, they do not provide a complete solution for repository understanding, architecture explanation, project summarization, bug investigation, documentation generation, change planning, and safe code modification in a unified workflow.

Developers also struggle with navigation, documentation, review, and architecture understanding in large systems, often switching between multiple tools.

8 Positioning of RepoMind AI

RepoMind AI is related to tools such as GitHub Copilot, Cursor, Cody, Continue, Tabby, Aider, and OpenHands. However, its focus is different.

It is a **learning-oriented, repository-aware assistant** that provides:

- Repository import and indexing
- Conversational repository-aware chat
- Architecture explanation
- Bug investigation
- Feature planning and change preparation
- Documentation generation
- Code review with diff preview
- Approval-based safe editing

The core idea behind RepoMind AI is **Retrieval-Augmented Generation (RAG)** at repository level, where code context is retrieved first and then used to generate grounded responses.

Chapter 2

Methodology/Proposed System Plan

1 Repository Input

RepoMind AI is designed as a step-by-step system that helps users understand, explore, document, and improve a software repository with the help of AI. The main idea is to first collect information from the repository and then use that information to answer questions, create documentation, plan changes, review code, and help with commit messages. [11]

2 File Scanning and Filtering

After the repository is imported, the system scans all project files. It ignores unnecessary files such as `.git`, `.env`, `node_modules`, `build` folders, `binary` files, and other sensitive files. This helps the system

3 Code Chunking

Large files are divided into smaller parts called chunks. Each chunk contains a section of code or documentation along with its file path and location. This helps the AI find and understand the required information quickly without reading the entire project every time. [12]

4 Embedding and Indexing

The system converts each chunk into a searchable format using an embedding model. These embeddings are stored in a vector database. This helps the system quickly find the most relevant files or code sections when the user asks a question. [5]

5 Repo-Aware Chat

When the user asks a question, the system searches for related code chunks from the repository. These chunks are given to the AI as context, and the AI generates an answer based on the actual project content. The answer also includes source references so that users can check where the information came from. [1]

6 Repository Navigation

The system helps users find where a feature, function, or API is implemented in the repository. It searches the project files and explains which files or modules are responsible for a specific task. [6]

7 Architecture Understanding

The system studies the repository structure and groups files into different sections such as frontend, backend, database, testing, configuration, and documentation. This helps create architecture summaries, file structure views, and system diagrams.

8 Bug Investigation

For bug-related questions, the system reads the bug description and searches the related files. It finds possible causes of the problem and suggests where the issue can be fixed. The system only provides suggestions and does not change the code automatically. [9]

9 Change Planning

Before making any changes, the system creates a plan. The plan shows which files may need to be added, edited, or removed, why the changes are needed, possible risks, and suggested tests. This allows the user to review everything before making any changes.

10 Approval-Based Editing

If the user approves the plan, the system prepares the required changes and shows a preview of the modifications. The user can review the changes before applying them. No changes are made automatically without the user's approval.

11 Review and Testing Guidance

After the changes are applied, the system reviews the updated files. It checks for possible bugs, missing validations, risky code, and missing tests. The

system can also suggest test commands, but the user decides whether to run them. [2]

12 Documentation Generation

The system can create project documentation such as README files, architecture summaries, setup guides, API documentation, and onboarding guides. The documentation is generated using the actual repository content and structure.

13 Commit and PR Assistance

After the user finishes making changes, the system generates a commit message, pull request (PR) title, and PR description based on the code changes. The final commit or push action is performed only after the user's approval.

14 Overall Workflow

The complete workflow of RepoMind AI is given below:

1. Import the Repository
2. Scan and Filter Files
3. Divide Code and Documentation into Chunks
4. Create Embeddings
5. Store Data in the Vector Database
6. User Asks a Question or Requests a Task
7. Retrieve Related Repository Information
8. AI Generates an Answer, Plan, Documentation, or Review
9. User Reviews the Result
10. If Changes Are Needed, Show a Preview and Ask for Approval
11. Apply Approved Changes
12. Review Changes, Generate Documentation, and Prepare Commit Support

15 Methodology Summary

The proposed method is to develop RepoMind AI as a safe and repository-aware AI assistant. Instead of relying only on general AI knowledge, the system first understands the repository by scanning files, creating chunks, and storing repository information. It then uses this information to answer questions, help users navigate the project, generate documentation, plan changes, review code, and support safe editing. The user remains in control throughout the entire process.

16 Overall Work Flow of the Project

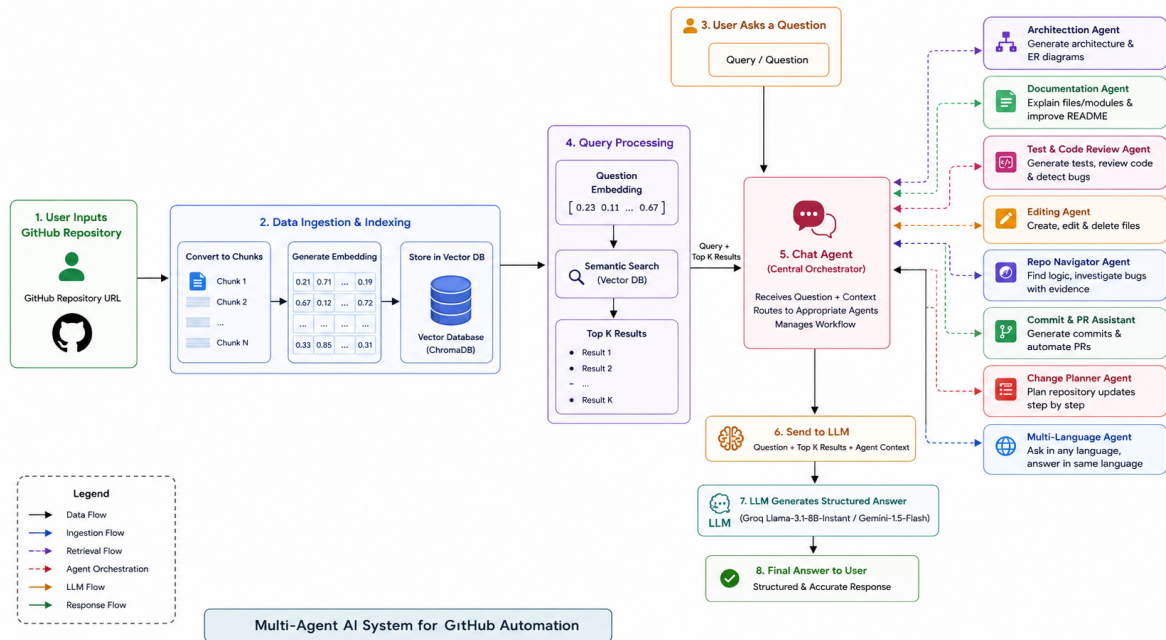


Figure 1: Work Flow of the Project [1]

Chapter 3

Full Development & Technical Architecture

1 Repository Input

Welcome to the **RepoMind AI Development & Architecture Document**. This document explains the design and working of RepoMind AI in simple and easy-to-understand language. It is prepared to help readers and presenters understand how the different components of the system work together. [1]

RepoMind AI is an intelligent coding assistant that understands the structure and content of a software repository. It allows developers to import code repositories, ask questions about the code, create plans for code changes, and apply those changes safely with human approval. It also helps review code differences (diffs) and automatically generates meaningful commit messages, making software development faster and more efficient. [13]

2 High-Level Technology Stack

RepoMind AI is built with a modern, decoupled architecture.

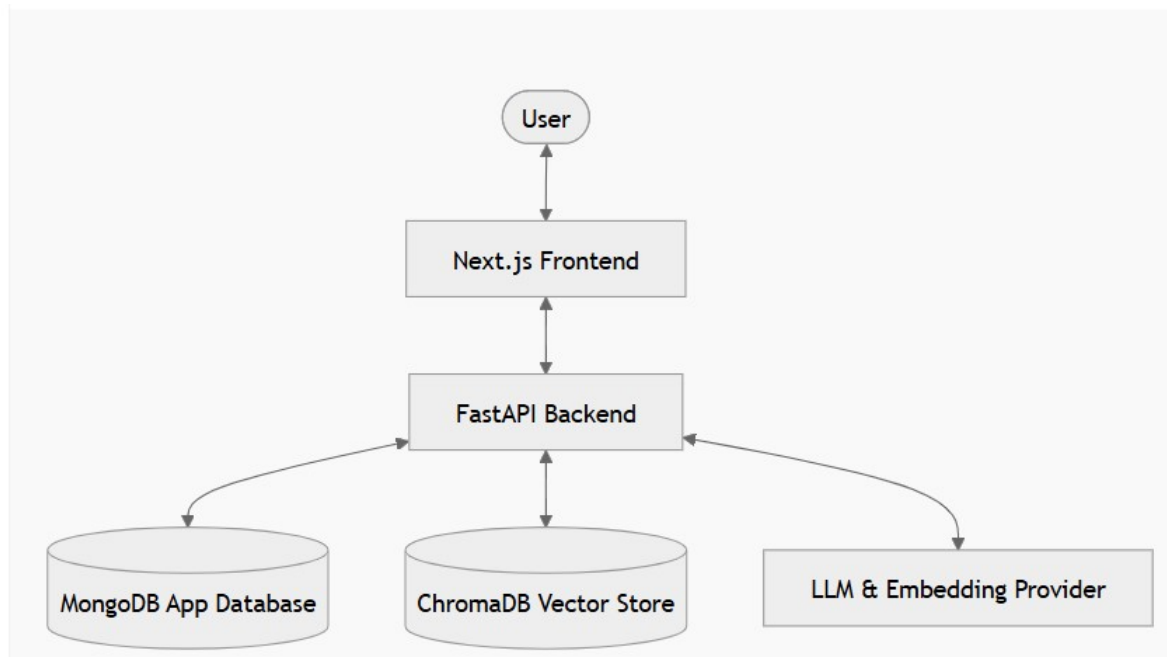


Figure 1: System Architecture [3]

2.1 Frontend (The User Interface)

- **Next.js (React Framework):** Used to build a fast, single-page interactive interface.

- **TypeScript:** Ensures code safety and fewer bugs during development.
- **Tailwind CSS:** Provides clean, modern styling (with dark mode and responsive layouts).

2.2 Backend (The Business Logic)

- **FastAPI (Python Framework):** A high-performance web framework used to build REST APIs. It is very fast and automatically documents APIs using Swagger.
- **Python:** Enables easy integration with data processing tools and LLM libraries.

2.3 Database & Storage (The State Management)

- **MongoDB (NoSQL Database):** Stores application state, project metadata, user sessions, chat sessions, and message history.
- **ChromaDB (Vector Database):** Stores code chunks and their mathematical meanings (vector embeddings) to allow quick semantic search.
- **Local Storage (./data/repos):** Stores cloned Git repositories locally for analysis and editing.

2.4 LLM & Embedding Layer (The Intelligence)

- **Local Options:** Ollama runs LLMs (like Llama 3) and embedding models (like `nomic-embed-text`) locally on computer.
- **Cloud APIs:** Support for Gemini and Groq for high-speed cloud-based reasoning.

3 System Components & Responsibility

Here is what each backend component does:

Component	Responsibility	Relevant Files
API Routers	Handles API requests from the frontend, manages authentication, and routes tasks.	<code>chat.py</code>
File Scanner	Scans directories, filters out ignored files (like <code>.git</code> and <code>node_modules</code>), and splits code into chunks.	<code>file_scanner.py</code>
Embedding Provider	Translates raw code chunks into vector lists using either a local hash function or Ollama/Cloud models.	<code>embedding_provider.py</code>
Vector Store	Connects to ChromaDB to save code vectors and retrieve relevant code when a user asks a question.	<code>vector_store.py</code>
Specialist Agents	AI sub-services focused on specific tasks (Planning, Investigating, Reviewing, Documenting).	<code>planner_service.py</code> , <code>investigation_service.py</code>
Editing Tools	Performs the actual file creations, edits, and deletions after human approval.	<code>editing_tools.py</code>
Commit Assistant	Inspects current git modifications, drafts clear commit messages, and handles the local Git commits.	<code>commit_assistant_service.py</code>

Table : Components & Responsibility [12]

4 Comprehensive Agent Implementation & Algorithm

This part provides a professional, deep technical breakdown of the **AI Agent Scaffolding & Code-Level Algorithms** in RepoMind AI. It explains the exact Python logic, regular expression (regex) processing, static analysis checks, and Git integrations that govern each agent.

4.1 System Orchestration & Multi-Agent Routing

To maintain low latency and prevent LLM context clutter, RepoMind AI uses an **Orchestrator–Router Pattern** inside `chat.py`.

The Routing Architecture

Instead of forwarding all user queries to a single generic prompt, the system inspects incoming messages deterministically. The router determines the developer’s intent and directs the request to the appropriate specialist service. [10]

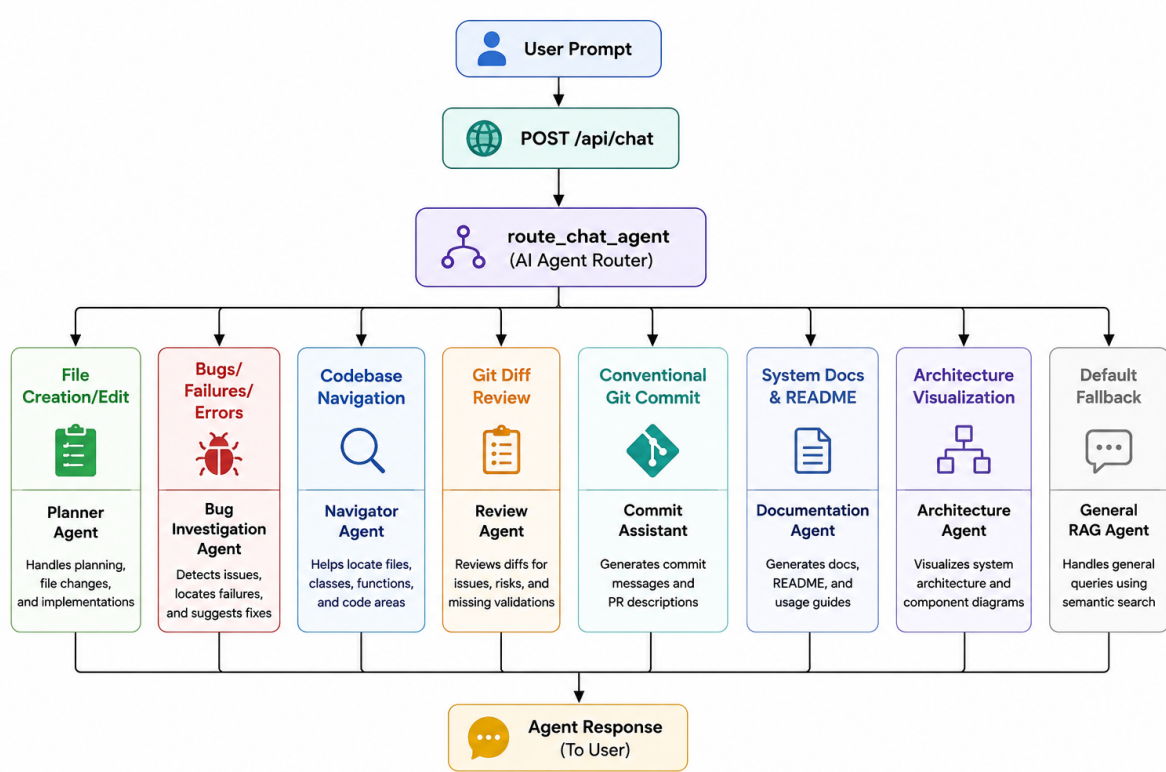


Figure 2: AI Agent Router Architecture [14]

The Routing Algorithm (`route_chat_agent`)

The router uses keywords and patterns in multiple languages to understand the user's request and direct it to the appropriate specialist agent.

```
def route_chat_agent(message: str) -> str:
    lowered = message.lower()

    if is_architecture_view_request(message):
        return "architecture"

    if is_readme_file_request(message):
        return "readme_editor"

    if is_file_change_request(message):
        return "planner"

    if is_documentation_request(message):
        return "documentation"

    if contains_any_term(
        message,
        MULTILINGUAL_REVIEW_TERMS
    ):

```

```

        return "review"

    if contains_any_term(
        message,
        MULTILINGUAL_COMMIT_TERMS
    ):
        return "commit"

    if contains_any_term(
        message,
        MULTILINGUAL_BUG_TERMS
    ) or "does nothing" in lowered:
        return "bug_investigation"

    if contains_any_term(
        message,
        MULTILINGUAL_NAVIGATION_TERMS
    ):
        return "navigator"

    return "chat"

```

4.2 Agent-by-Agent Implementation

Planner Agent (`planner_service.py`)

Role: Analyzes natural language requests for file edits or creations, compiles codebase evidence, and produces structured change plans (Proposed Operations) before editing files on disk.[2]

Sequence Diagram

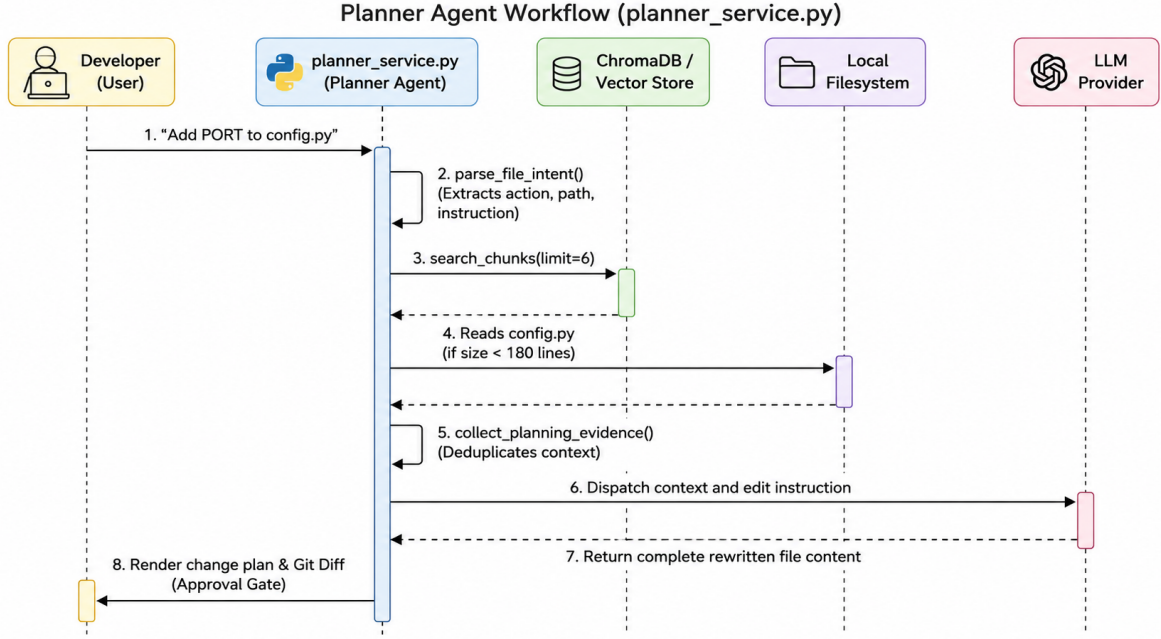


Figure 3: Planner Agent Sequence Diagram [2]

Code-Level Intent Parsing (`parse_file_intent`)

The agent extracts details from the user prompt using regular expression (regex) patterns.

1. Action Extraction:

Scans the prompt for keywords such as `create`, `edit`, `delete`, `replace`, or `remove` to determine the requested operation.

2. Path Extraction:

Evaluates file path formats (e.g., `path/to/file.ext` or filename with folder keywords) using the following regex patterns:

```
explicit_path_patterns = [
    r"(?:file\s+path|filepath|path)\s*[:=]\s*'?([A-Za-z0-9_./\+\-]+\.[A-Za-z0-9+])'?",
    r"['\"]([A-Za-z0-9_./\+\-]+\.[A-Za-z0-9+])['\"]",
]
```

3. Validation Check:

If critical parameters are missing (for example, when the action is `create` but no file path is provided), the agent requests the missing information from the developer before proceeding.

```
if action == "create" and not path:
    missing_fields.append("file path")
```

Evidence Gathering Algorithm (`collect_planning_evidence`)

To build context for the LLM without exceeding context limits, the agent performs a multi-stage search strategy.

1. **Dense Semantic Search:**

Retrieves up to 6 vector chunks that best match the user’s query using semantic similarity search.

2. **Keyword Matching:**

Filters out common stopwords, extracts up to 6 important semantic keywords, and searches code references for up to 2 matching results per keyword.

3. **Explicit File Parsing:**

If the user explicitly references a specific file, the agent reads the complete contents of that file directly from disk.

4. **Small File Context:**

If a matched file contains fewer than 180 lines, the agent loads the entire file and includes it in the context provided to the LLM.

5. **Deduplication and Context Capping:**

The agent merges all collected sources, removes overlapping or duplicate ranges, and limits the final context to a maximum of 20 chunks to avoid exceeding the LLM context window.

Code Generation and Fallback

1. **Editing Engine:**

The agent instructs the LLM to rewrite the target file and return only the updated file contents using the following prompt:

```
Rewrite the complete file content to satisfy the
user’s requested edit. Return only the full updated
file content. Do not include markdown fences,
explanations, or diffs.
```

2. **Rule-Based Fallbacks:**

If the LLM request fails, the service performs localized string replacement operations (for example, replacing one pattern with another) to generate the required result without depending on network connectivity.

Bug Investigation Agent (`investigation_service.py`)

Role: Diagnoses compilation and logic bugs by applying deterministic static rules before querying the LLM.[5]

Sequence Diagram

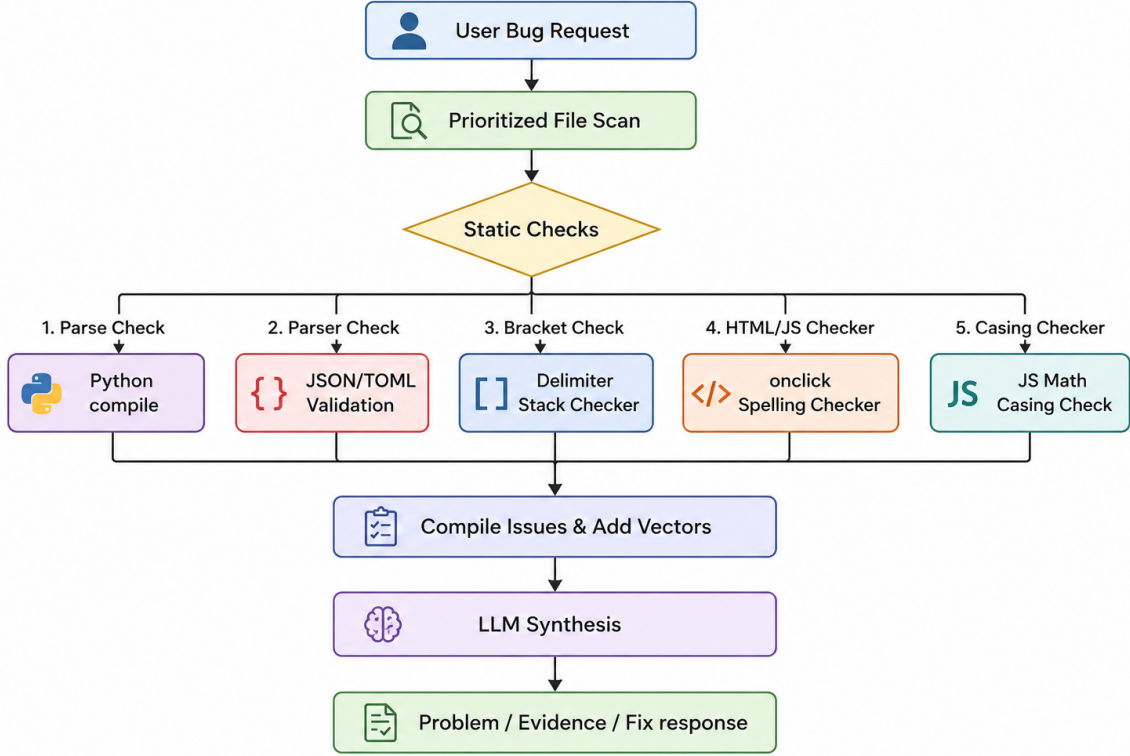


Figure 4: Bus Investigation Agent Sequence Diagram [5]

Prioritized Code Scanning (`_prioritized_bug_files`)

When a bug is reported, the agent scans up to 120 files, prioritizing files that are most likely to contain errors (such as `main.js`, `app.py`, and `page.tsx`) over configuration files, stylesheets, and documentation.

Extension Category	Priority Rank	Example Files
Logic & Scripts	Priority 0 (Highest)	.py, .js, .tsx, .html
Data & Configuration	Priority 1	.json, .toml, .yaml
Styling & Scripts	Priority 2	.css, .scss, .sql, .sh
Documentation	Priority 4 (Lowest)	.md, .txt, .log

Deterministic Static Diagnostic Rules

Before querying the LLM, the agent executes deterministic static analysis rules to detect syntax and logic errors.

1. Syntax Compiler Checks

Python: Validates Python files using the built-in compiler:

```
compile(file_content, file_path, "exec")
```

JSON/TOML: Validates structures using `json.loads()` and `tomllib.loads()` to detect trailing commas, missing quotes, and invalid syntax.

2. Delimiter Stack Algorithm

The system traces unmatched brackets using a character stack.

```
def verify_delimiters(content: str) -> list[str]:
    stack = []
    matching = {"(": ")", "{": "}", "[": "]"

    for char in content:
        if char in matching.values():
            stack.append(char)
        elif char in matching.keys():
            if not stack or stack.pop() != matching[char]:
                return ["Syntax Error: Unmatched bracket found"]

    return []
```

3. JavaScript Casing Checks

Detects incorrect lowercase JavaScript math calls such as `math.floor()`, which cause runtime errors.

```
re.search(
    r"\bmath\.(floor|random|ceil|round|max|min|abs)\b",
    line
)
```

4. JavaScript Syntax Validation

The agent detects invalid JavaScript syntax such as:

- Missing arrow operator in event listeners:

```
addEventListener('click', () {
```

- Incorrect statements like:

```
tr {
```

instead of

```
try {
```

5. HTML/JavaScript Cross-Reference Validation

The agent scans HTML files for `onclick` attributes such as:

```
onclick="changeColor()"
```

It then verifies whether the corresponding JavaScript function is defined in the associated script files. If the function is missing or misspelled, the system raises a warning.

Navigator Agent (*investigation_service.py* – *Navigator Mode*)

Role: Scans large codebases to help developers locate files, symbols, routes, or configurations.

The Navigation Algorithm

The navigation algorithm helps RepoMind AI locate relevant code within the repository and present the results in a structured manner. It consists of the following steps: [4]

1. Token Extraction

The user query is divided into meaningful search tokens for efficient processing.

2. Symbol Lookup

The system searches the repository index for matching classes, functions, variables, or filenames.

3. Semantic Context Matching

The query is converted into an embedding vector and compared with indexed code chunks to find the most semantically similar results.

4. Response Construction

The retrieved matches are organized into a structured response containing relevant files, line references, and brief explanations. The Navigator Agent follows the instruction:

```
"You are RepoMind's Navigator Agent.  
Summarize where the requested feature is  
located in the codebase.  
Capped at 180 words."
```

Review Agent (`review_service.py`)

Role: Reviews unstaged/uncommitted Git changes to catch syntax errors, debugging statements, and security issues, and suggests testing commands. [7]

Sequence Diagram

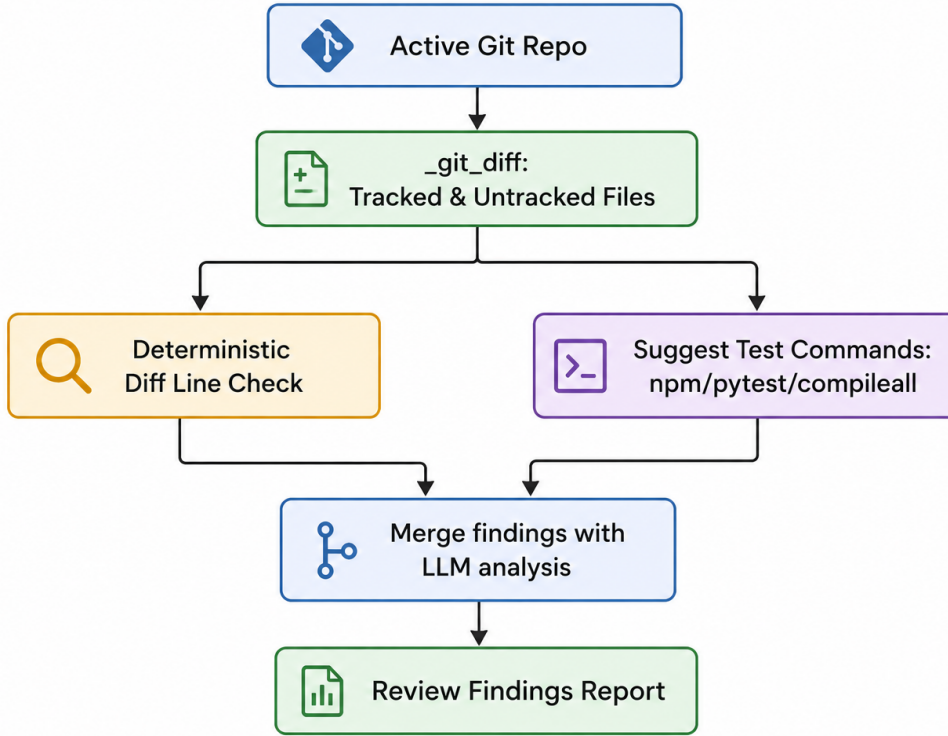


Figure 4: Review Agent Sequence Diagram [7]

Diff Extraction and Diagnostic Engine

1. Diff Extraction Engine (`_git_diff` & `_untracked_file_diff`)

The agent extracts tracked changes using GitPython through `repo.git.diff("-", ".")`. To include newly created files that are not yet tracked by Git, the agent generates a mock unified diff against `/dev/null`, ensuring that untracked files are also reviewed.

```
unified_diff(  
    [],  
    new_file_lines,  
    fromfile="/dev/null",  
    tofile=f"b/{normalized_path}"  
)
```

2. Deterministic Diff Diagnostics (`_deterministic_findings`)

The agent examines all added lines in the diff (lines beginning with `+`) to detect common programming errors.

- **Merge Conflict Markers**

Checks for unresolved Git conflict markers such as

```
<<<<<<< HEAD
=====
>>>>>>> branch
```

- **Incorrect Callbacks**

Detects invalid JavaScript callback syntax, such as missing arrow notation in event listeners.

- **Case Sensitivity Errors**

Flags incorrect lowercase calls like `math.floor()` or `.random()` that should use the JavaScript `Math` object.

3. Dynamic Test Discovery (`_suggest_test_commands`)

The agent analyzes project manifest files in the root directory and suggests appropriate test commands automatically.

- **Node.js (`package.json`)**

Parses available scripts and suggests commands such as:

```
npm test
npm run lint
npm run build
```

- **Python (`pyproject.toml` / `requirements.txt`)**

Suggests suitable Python test commands such as:

```
python -m pytest
python -m compileall app
```

Commit Assistant (`commitassistantservice.py`)

Role: Auto-generates conventional commit messages and Pull Request markdown descriptions, stages changes, and pushes commits to remote repositories. [8]

Sequence Diagram

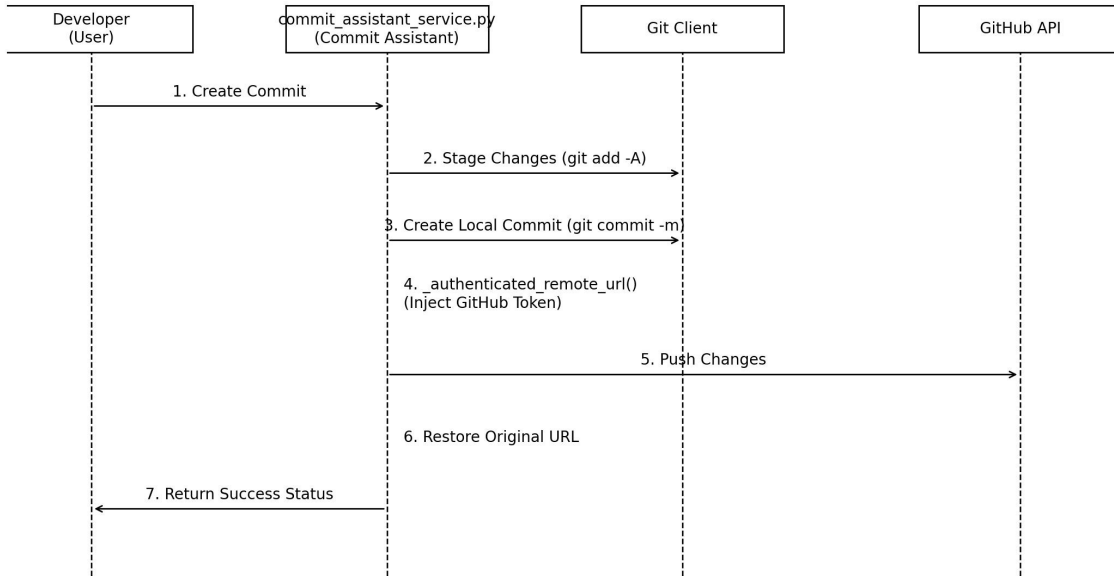


Figure 4: Commit Assistant Sequence Diagram [8]

Suggestion Parsing (`_parse_suggestion`)

The agent asks the LLM to generate a conventional commit message and a Markdown pull request (PR) body. The output is then parsed into structured fields.

`COMMIT_MESSAGE`: feat: add user authentication

`PR_TITLE`: Implement User Authentication

`PR_DESCRIPTION`:

Summary

- Added password hashing in `security.py`
- Added login validation
- Updated authentication middleware

Secure Remote Authentication (`_authenticated_remote_url`)

To push commits to remote repositories, the agent injects the developer's temporary GitHub OAuth token into the remote URL configuration dynamically.

1. HTTPS URLs

The authorization token is inserted into the repository URL:

```
https://github.com/owner/repo
->
https://x-access-token:{token}@github.com/owner/repo
```

2. SSH URLs

SSH repository URLs are converted into authenticated HTTPS URLs:

```
git@github.com:owner/repo.git
->
https://x-access-token:{token}@github.com/owner/repo.git
```

Cleanup

After the push operation completes, the agent restores the original unauthenticated repository URL to ensure that authentication tokens are not stored in the local Git configuration.

Error Masking and Sanitization (`_sanitize_git_error`)
5[mm]

If a Git operation fails, the agent sanitizes the error output before displaying it to the user. Any sensitive authentication tokens are removed from the logs.

```
def _sanitize_git_error(details: str, token: str) -> str:
    cleaned = details.replace(token, "[redacted]")
    cleaned = re.sub(
        r"https://x-access-token: [^\s]+@",
        "https://[redacted]@",
        cleaned
    )
    return cleaned
```

This mechanism ensures that authentication credentials are never exposed in error messages, logs, or user-visible outputs.

Architecture Agent (`chat.py`)

Role: Opens the Architecture workspace so developers can visualize and navigate the codebase structure dynamically as an interactive map.

Routing Intent Detection (`is_architecture_view_request`)

The orchestrator analyzes the user's message and selects the Architecture Agent only when specific conditions are satisfied.

Exclusion Check

If the request contains file modification terms such as `add`, `create`, `edit`, `modify`, `update`, `write`, `generate`, or `replace`, the request is not routed to the Architecture Agent. Similarly, requests mentioning `README` or `README.md` are redirected to specialized agents such as the Planner Agent or README Agent.

Term Check

The orchestrator scans the message for keywords defined in the `MULTILINGUAL_ARCHITECTURE_TERMS` collection. Examples include:

- architecture
- file structure
- folder structure
- diagram
- ERD
- data model

View Trigger

The request must contain a visualization-related keyword from the `MULTILINGUAL_ARCHITECTURE_VI` collection, such as:

- view
- show
- open
- visualize
- overview
- diagram

Alternatively, the request may consist solely of an architecture-related term.

Workspace Redirection and Focus Map Handoff

Unlike most agents that generate responses through LLM text output, the Architecture Agent primarily performs workspace routing.

Direct Response

The agent returns a predefined response that guides the user to the architecture workspace:

“I opened the Architecture workspace so you can inspect the repository as an interactive map. Use the focus controls to switch between overview, frontend, backend, and data-oriented layers.”

API State Routing

The FastAPI backend updates the response payload to trigger a client-side workspace redirection.

```
suggested_workspace_mode
->
"architecture"

agent_status
->
"redirect_required"
```

This mechanism allows the user interface to automatically switch from the current workspace to the Architecture workspace, enabling repository exploration through an interactive architectural map rather than a standard text-based response.

5 Step-by-Step Technical Flows

5.1 Flow A: Ingesting & Indexing a Repository

When you add a new GitHub repository, the backend does the following:

Sequence Diagram

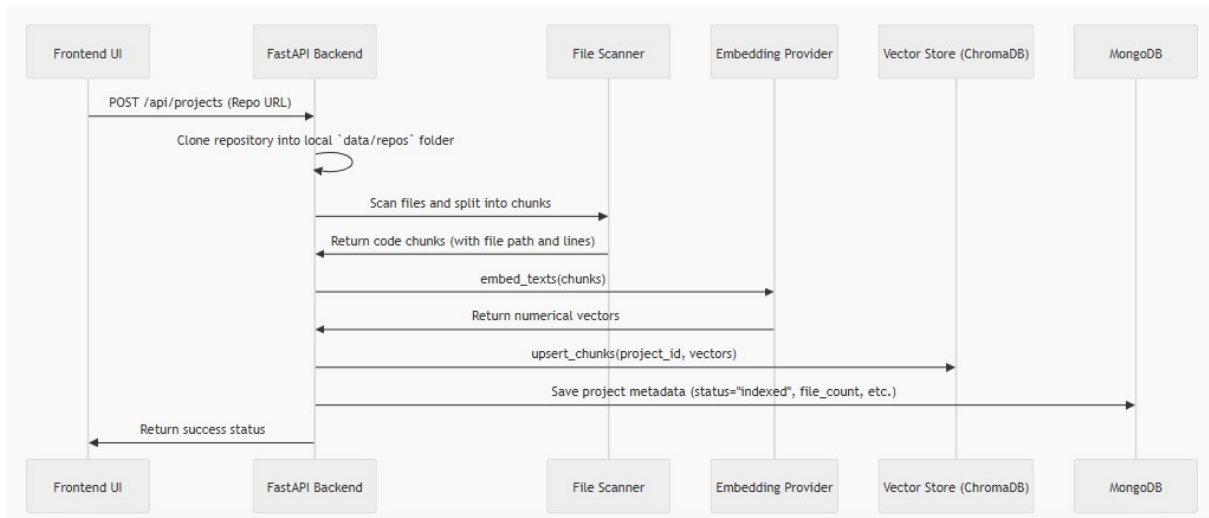


Figure 5: Ingesting & Indexing a Repository [1]

1. **Repository Ingestion:** The backend clones the repository into the `backend/data/repos` directory.
2. **Code Scanning:** The `file_scanner.py` module parses the repository. It skips binary files, node packages, and Git configuration files.
3. **Chunking:** The files are divided into small, manageable chunks of code (for example, 30 lines per chunk).
4. **Embedding Generation:** Each chunk is converted into a vector representation (embedding) using the `embedding_provider.py` module.

5. **Vector Database Insertion:** The code chunks and embeddings are stored in ChromaDB, organized under a collection unique to the corresponding project.
6. **MongoDB Metadata Update:** MongoDB stores project-related metadata such as the project name, repository path, processing state, and indexing information.

5.2 Flow B: User Chat and Smart Agent Routing

When a user types a message in Chat, RepoMind does not just blindly send it to the LLM. It routes the message to a Specialist Agent based on the request's intent:

Sequence Diagram

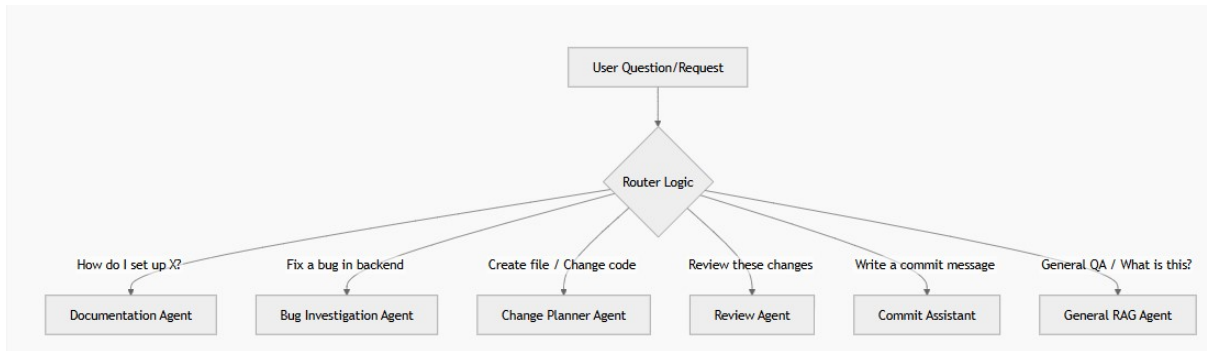


Figure 5: Smart Agent Routing [8]

1. **Query Analysis:** The message is parsed in `chat.py` using the `route_chat_agent` router.
2. **Specialist Selection:**
 - **Planner Agent:** Handles requests containing terms such as *add*, *modify*, *create*, or *delete*.
 - **Bug Investigation Agent:** Handles requests containing terms such as *bug*, *error*, *fail*, or *does not work*.
 - **Review Agent:** Selected when the user requests a code review or asks to inspect differences.
 - **Commit Assistant:** Triggered for commit message generation or pull request related requests.
 - **General RAG:** Performs semantic search and repository-aware retrieval to answer general questions.

5.3 Flow C: Safe Code Editing and Git Commit (The Safety Loop)

Editing files is a critical operation. RepoMind AI enforces a strict Safety Loop to ensure that the AI never modifies code without explicit user consent.

Sequence Diagram

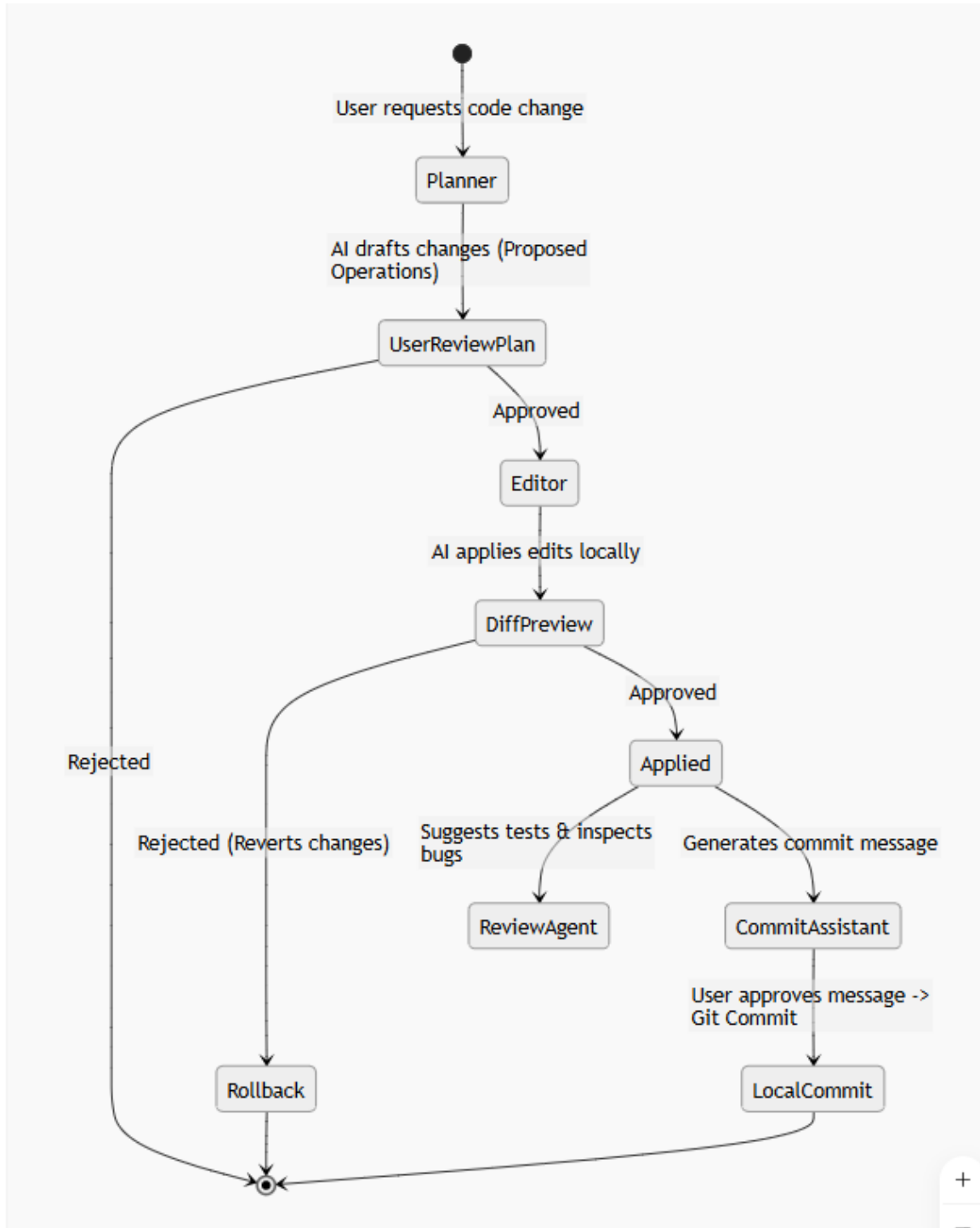


Figure 5: Safe Code Editing & Git Commit [13]

1. **Planning:** The Planner Agent explains which files it intends to create, modify, or delete, along with the reasons for each proposed change.
2. **Handoff Approval:** The user is presented with a list of planned file modifications in the user interface and must explicitly click *Approve* before any changes are made.
3. **Local Editing:** After approval, the backend performs file operations using the `create_file`, `edit_file`, or `delete_file` functions available in `editing_tools.py`.

4. **Diff Preview:** The frontend displays a color-coded code difference view, where additions are highlighted in green and deletions are highlighted in red.
5. **Rollback Option:** If the user is not satisfied with the proposed modifications, they can select *Reject*. The backend then restores the previous state of the repository using Git rollback mechanisms.
6. **Apply and Review:** If the user selects *Apply*, the changes become part of the working repository, and the Review Agent recommends appropriate testing commands and validation steps.
7. **Commit Assistant:** The Commit Assistant analyzes the accepted modifications, generates an appropriate commit message, and creates a local Git commit.

6 Result & Discussion

Result 1 :

Scenario: The user asks:

“Please add a PORT configuration parameter to our `config.py`.”

Step 1: User Input and Routing

- **User Action:** The user enters the prompt into the Next.js chat interface.
- **Routing Logic:** The API endpoint `POST /api/chat` receives the message.
- **Detection:** The `route_chat_agent` function detects keywords such as *add* and `config.py`, and routes the request to the Planner Agent.

Step 2: Code Retrieval (RAG)

- **Vector Search:** The backend executes:

```
search_chunks(  
    project_id,  
    "add a PORT configuration parameter to our config.py"  
)
```

- **Result:** ChromaDB returns the contents of `backend/app/core/config.py`.

Step 3: Change Planning

- **LLM Processing:** The Planner Agent receives the contents of `config.py` as contextual information.
- **Output:** The agent generates a structured change plan:

```
[
  {
    "action": "edit",
    "path": "backend/app/core/config.py",
    "description":
      "Add PORT variable with default value 8000"
  }
]
```

- **UI Rendering:** The frontend displays an interactive approval card asking:

“Would you like to edit config.py to add PORT?”

Step 4: Applying Edits Safely

- **User Action:** The user clicks *Approve*.
- **AI Edit:** The backend loads `config.py`, identifies the settings class, and inserts:

```
port: int = 8000
```

- **Diff Preview:** The frontend presents a side-by-side comparison showing the modifications made to `config.py`.

Step 5: Committing the Work

- **User Action:** The user clicks *Apply & Commit*.
- **Diff Analysis:** The Commit Assistant executes `git diff` and detects:

```
+ port: int = 8000
```

- **Message Generation:** The LLM generates the commit message:

“feat: add port configuration to application settings”

- **Git Execution:** The backend executes:

```
git commit -am
"feat: add port configuration to application settings"
```

Result 2 :

Scenario: The developer asks:

“Why is the background color change not working when I click the button?
It does nothing.”

Step 1: User Input and Routing

- **User Action:** The developer enters the issue description into the chat interface.
- **Routing Logic:** The API endpoint receives the message and forwards it to the routing layer.
- **Detection:** The `route_chat_agent` function scans the query and matches phrases such as *not working* and *does nothing* against the `MULTILINGUAL_BUG_TERMS` collection.
- **Agent Selection:** The request is routed to the Bug Investigation Agent (`investigate_codebase` service in `investigation_service.py`).

Step 2: Deterministic File Scan and Parsing

- **Repository Scan:** The backend analyzes repository files and prioritizes logic files such as `index.html` and `main.js`.
- **Static Analysis:** The file contents are parsed line-by-line using the `_detect_bug_issues` function.
- **Finding 1 (Case Sensitivity Error):** The analyzer flags line 45 containing:

```
math.random()
```

The regular expression rule detects the incorrect lowercase usage of `math` and registers a `bad_math_case` issue.

- **Finding 2 (Syntax Error):** The analyzer flags line 48 containing:

```
button.addEventListener("click", () {
```

The regular expression rule identifies a missing arrow operator (`=>`) in the event handler and registers a `bad_click_handler` issue.

Step 3: Context Construction

- **Issue Aggregation:** The detected warnings are collected into a `detected_issues` list.
- **Knowledge Retrieval:** The system retrieves similar code examples and related implementation patterns from ChromaDB.
- **Context Preparation:** All findings are combined into a structured debugging context.

Step 4: Structured Output Generation

- **Deterministic Processing:** Since the issues are detected through predefined rules, the service bypasses the LLM and generates the response directly.

- **Problem Identified:**

```
Case sensitivity error and syntax error  
in click handler.
```

- **Evidence:**

```
index.html:45 -> math.random()  
index.html:48 -> () {
```

- **Recommended Fix:**

```
Replace math.random() with Math.random()
```

```
Add the => operator:
```

```
button.addEventListener("click", () => {  
    ...  
})
```

Step 5: Final Response Delivery

- **Result Generation:** The Bug Investigation Agent formats the findings into a user-friendly report.
- **User Output:** The developer receives the identified issues, supporting evidence, and recommended fixes for resolving the background color change problem.

Result 3:

Scenario: The developer asks:

“Please review my changes before I commit.”

Step 1: User Input and Routing

- **User Action:** The developer submits a review request through the chat interface.
- **Routing Logic:** The orchestrator analyzes the message and searches for review-related keywords.
- **Detection:** The router detects the keyword *review* from the `MULTILINGUAL_REVIEW_TERMS` collection.
- **Agent Selection:** The request is routed to the `review_changes` service in `review_service.py` with `agent = "review"`.

Step 2: Workspace Mapping

- **Workspace Resolution:** The function `workspace_for_agent("review")` returns the workspace mode "review".
- **Frontend Update:** The API response includes:

```
suggested_workspace_mode = "review"
```

which automatically switches the user interface to the Review Workspace.

Step 3: Active Diff Extraction

- **Git Analysis:** The Review Agent uses GitPython to extract repository differences:

```
repo.git.diff("--", ".")
```

- **Untracked File Detection:** The system discovers an untracked file:

```
src/helper.js
```

- **Unified Diff Generation:** The file is compared against `/dev/null`, and its contents are appended to the unified diff stream.

Step 4: Static Diff Review Checks

- **Code Inspection:** The agent scans newly added lines (prefixed with `+`) in the generated diff.
- **Finding 1:** A leftover debugging statement is detected:

```
console.log(response)
```

- **Finding 2:** An empty `try-catch` block is identified, indicating incomplete error handling.
- **Warning Registration:** These findings are added to the list of review warnings.

Step 5: Dynamic Test Suggestion

- **Project Inspection:** The Review Agent examines the repository root directory.
- **Configuration Detection:** It locates the `package.json` file and parses the available scripts.
- **Recommended Commands:** The system identifies the following validation commands:

```
npm test  
npm run lint
```

- **Suggestion Generation:** These commands are appended to the recommended actions list.

Step 6: Response Construction

- **LLM Analysis:** The complete diff is provided to the LLM for high-level logic review and summarization.
- **Result Aggregation:** The LLM findings are combined with the deterministic static-analysis warnings.
- **Final Output:** The response is organized into the following sections:
 - Review Findings
 - Tests To Add Or Update
 - Suggested Test Commands
 - Residual Risks

Step 7: Final Review Report

- **Result Delivery:** The developer receives a comprehensive review report containing code-quality observations, potential risks, and recommended validation steps before committing the changes.
- **Outcome:** The review process helps identify issues early and improves code quality before the changes are committed to the repository.

Result 4:

Scenario: The developer approves the changes and clicks:

“Commit and Push Changes”

with the commit message:

“feat: add user authentication”

Step 1: Security Authorization Check

- **Access Validation:** The system invokes the `ensure_project_write_access()` function.
- **Permission Verification:** The function verifies that the repository is not configured as read-only.
- **Token Validation:** The system confirms that a valid GitHub access token is available in MongoDB before proceeding with any write operation.

Step 2: Staging and Committing Changes

- **File Staging:** The Commit Assistant stages all modified files using GitPython:

```
repo.git.add("-A")
```

- **Local Commit Creation:** The system creates a local commit using:

```
repo.index.commit(  
    "feat: add user authentication"  
)
```

- **Result:** All approved changes are safely recorded in the local Git repository.

Step 3: Dynamic Remote Authentication

- **Remote URL Retrieval:** The system fetches the repository's remote URL:

```
https://github.com/owner/repo.git
```

- **Token Injection:** The `_authenticated_remote_url()` function dynamically injects the user's OAuth access token:

```
https://x-access-token:  
gho_TOKEN123@github.com/owner/repo.git
```

- **Temporary Configuration:** The Git remote is temporarily updated to use the authenticated URL for the push operation.

Step 4: Git Push and Error Sanitization

- **Push Execution:** The system executes:

```
remote.push(refspec="main:main")
```

- **Failure Handling:** If the push operation fails, the generated error message is passed to the `_sanitize_git_error()` function.
- **Credential Protection:** The sanitization process replaces sensitive tokens such as:

```
gho_TOKEN123
```

with:

```
[redacted]
```

- **Log Security:** This prevents authentication credentials from appearing in application logs or user-visible error messages.

Step 5: Cleanup and Remote Restoration

- **Successful Push:** If the push operation succeeds, the system immediately restores the original remote URL.

`https://github.com/owner/repo.git`

- **Credential Removal:** Any temporary authentication information is removed from the Git configuration.
- **Security Guarantee:** No OAuth tokens remain stored in local Git configuration files after the operation completes.

Step 6: Final Result

- **Commit Completed:** The approved changes are committed and pushed successfully to the remote repository.
- **Secure Workflow:** Throughout the process, authentication credentials remain protected through temporary injection and automatic cleanup mechanisms.
- **Outcome:** The repository is updated while maintaining strong security and preventing accidental credential exposure.

Result 5:

Scenario: The developer asks:

“Show me the project database diagram map.”

Step 1: User Input and Routing

- **User Action:** The developer submits a request to view the project architecture.
- **Query Analysis:** The orchestrator scans the request for architecture-related keywords.
- **Detection:** The query contains the terms *diagram* and *map* from the `MULTILINGUAL_ARCHITECTURE` collection, along with the term *show* from the `MULTILINGUAL_ARCHITECTURE_VIEW_TERMS` collection.
- **Exclusion Check:** The routing system verifies that the request does not contain file-editing keywords such as *add*, *modify*, *create*, or *delete*.
- **Agent Selection:** The orchestrator returns:

`routed_agent = "architecture"`

Step 2: Static Action Short-Circuiting

- **LLM Bypass:** Since architecture-view requests do not require natural language generation, the system bypasses the LLM entirely.
- **Static Response Loading:** The Architecture Agent retrieves a predefined routing message:

“I opened the Architecture workspace so you can inspect the repository as an interactive map. Use the focus controls to switch between overview, frontend, backend, and data-oriented layers.”

- **Response Preparation:** The static message is prepared for delivery to the frontend.

Step 3: Client-Side Redirection Payload

- **Payload Generation:** The API constructs the following response payload:

```
{  
  "suggested_workspace_mode": "architecture",  
  "agent_status": "redirect_required"  
}
```

- **Frontend Detection:** The Next.js frontend detects the value of `agent_status`.
- **Workspace Switching:** The interface automatically switches to the Architecture Workspace.

Step 4: Architecture Visualization

- **Interactive View Loading:** The Architecture panel is opened automatically.
- **Repository Mapping:** The user is presented with an interactive node-based visualization of the repository structure.
- **Focus Controls:** The interface allows switching between different architectural perspectives:
 - Overview Layer
 - Frontend Layer
 - Backend Layer
 - Data-Oriented Layer

Step 5: Final Result

- **Workspace Redirection Completed:** The developer is automatically redirected to the Architecture Workspace.
- **Visual Exploration Enabled:** Instead of receiving a text response, the user can explore the repository structure through an interactive architecture diagram.
- **Outcome:** The repository architecture becomes easier to understand through visual navigation and layer-based exploration.

7 Architectural Highlight: Multi-Dimension Collection Isolation

One of the most innovative features of RepoMind AI is its ability to handle multiple embedding models while preventing conflicts within the vector database.

The Problem

When the embedding model is changed (for example, from a basic hash-based embedding system to a higher-quality model such as `nomic-embed-text`), the dimensionality of the generated vectors also changes. For instance, one model may generate vectors containing 384 dimensions, while another may generate vectors containing 768 dimensions.

Storing embeddings of different dimensions within the same database collection can lead to indexing errors, retrieval failures, and database inconsistencies.

The Solution in RepoMind AI

To address this issue, the `collection_name()` function in `vector_store.py` automatically generates a unique collection name for each project based on its embedding provider signature. [10]

```
def collection_name(project_id: str) -> str:
    safe_project_id = project_id.replace("-", "_")

    safe_embedding = re.sub(
        r"^[a-zA-Z0-9_]+",
        "_",
        embedding_signature()
    )

    if safe_embedding == "hash":
        return f"project_{safe_project_id}"

    embedding_hash = hashlib.sha256(
        safe_embedding.encode("utf-8")
    ).hexdigest()[:12]

    return f"project_{safe_project_id[:8]}_{embedding_hash}"
```

Benefits

- Prevents vector dimension conflicts between different embedding models.
- Ensures that each embedding provider uses an isolated collection.
- Improves database reliability and retrieval accuracy.
- Allows seamless migration between embedding models without affecting previously indexed repositories.

- Supports future expansion with additional embedding providers and vector dimensions.

By isolating collections according to embedding signatures, RepoMind AI maintains compatibility across multiple embedding models while ensuring stable and efficient vector database operations.

8 Key Presentation Takeaways (Bullet Points)

The following points can be highlighted during the project presentation to emphasize the unique strengths of RepoMind AI:

- **100% Secure and Controlled:** RepoMind AI never modifies source code directly. All proposed changes are presented to the user through a diff preview and require explicit approval before being applied. [2]
- **Decoupled and Flexible Architecture:** The system uses an abstract services layer, allowing developers to replace vector databases (e.g., ChromaDB with Pinecone) or LLM providers (e.g., Ollama with Groq or Gemini) without major architectural changes.
- **Intelligent Agent Routing:** The chat system acts as an orchestrator that analyzes user intent and routes requests to specialized agents, resulting in more accurate responses and reduced context clutter. [7]
- **Smart Repository Ingestion:** The file scanning mechanism automatically ignores large binary files, dependency packages, and unnecessary directories, ensuring that the vector database remains efficient, clean, and optimized for retrieval.

Chapter 4

Testing & System Transparency Document

Our System gives a clear and simple overview of the **RepoMind AI Testing Suite**. It explains how the system works, including how code is processed, searched, and understood by the AI. [5]

To make it easy to understand, we will follow a real example from the **Authentication Module** and see how data moves through the entire system, from the code repository to the final AI response. [6]

1 System Transparency & Data Flow (The Security Trace)

Here is a step-by-step explanation of how RepoMind AI ingests repository code, embeds it, retrieves it, feeds it to the LLM, and automates changes.

Step 1: Code Ingestion & Vector Conversion (Embeddings)

When a repository is imported, RepoMind scans it and chunks its files. Let us use a real function from `security.py` as our example:

```
# Raw code snippet from backend/app/core/security.py
def hash_password(password: str) -> str:
    pwd_bytes = password.encode("utf-8")
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(pwd_bytes, salt)
    return hashed.decode("utf-8")
```

1. **Scanning:** The file scanner reads `security.py` and extracts the above lines.
2. **Chunking:** The code is split into small pieces (chunks) with metadata:
 - **File Path:** `backend/app/core/security.py`
 - **Start Line:** 11 | **End Line:** 15
3. **Token Extraction:** The default hash embedding function extracts key terms (tokens) from the chunk:

```
["def", "hash_password", "password", "str",
 "pwd_bytes", "encode", "utf_8", "salt",
 "bcrypt", "gensalt", "hashed", "hashpw",
 "return", "decode"]
```

4. **Vectorization:** Every token is converted into a 384-dimensional mathematical vector (a list of 384 decimal numbers) using a deterministic SHA-256 hash mechanism:

```
[0.0512, -0.0123, 0.0891, ..., -0.0034]
```

5. **Storage:** This vector and its original code content are saved to ChromaDB.

Step 2: Question Embedding & ChromaDB Search

When a user asks a question, the system searches the vector database for relevant code.

- **User Question:**

“How does the system securely hash user passwords?”

- **Vector Query Process:**

- The user’s question is converted into a 384-dimensional vector using the same embedding model that was used for the repository code.
- ChromaDB compares this question vector with all the stored code vectors using a method called **Cosine Similarity**. It finds the code chunks that are most closely related to the question.
- **Result:** ChromaDB finds the matching code in `security.py` that contains the `hash_password` function and returns it for further processing by the AI.

```
{
  "file_path": "backend/app/core/security.py",
  "start_line": 11,
  "end_line": 15,
  "content": "def hash_password(password: str) -> str:\n
    pwd_bytes = password.encode(\"utf-8\")..."
}
```

Step 3: LLM Context Construction (The Prompt)

Once the code chunks are fetched, the backend constructs the exact context prompt that will be sent to the LLM. It packs the user query and retrieved code chunks into a structured template:

```
You are a repository-aware coding assistant.
You must answer the user’s question using only
the provided context. If you do not know the
answer based on the context, say so.
Do not make things up.
```

```
RELEVANT FILE: backend/app/core/security.py
LINES: 11-15
```

```
def hash_password(password: str) -> str:
    pwd_bytes = password.encode("utf-8")
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(pwd_bytes, salt)
    return hashed.decode("utf-8")
```

USER QUESTION:

How does the system securely hash user passwords?

YOUR ANSWER:

Step 4: LLM Answer Generation

The constructed prompt is dispatched to the configured LLM (such as a local Ollama Llama 3 model or the Gemini API).

- **LLM Processing:** The LLM reads the context block, identifies the use of `bcrypt` with `gensalt()` and `hashpw()`, and understands the encoding and decoding process.
- **Generated Answer Output:**

“The system hashes passwords using the `hash_password` function in `backend/app/core/security.py`. It encodes the plain-text password to UTF-8, generates a secure random salt using `bcrypt.gensalt()`, hashes the password with `bcrypt.hashpw()`, and finally decodes the resulting hash back to a UTF-8 string before returning it.”

Step 5: Agent Automation (How it Works)

When a user requests a code change (e.g., “*Change the password reset link expiration to 30 minutes*”), the system performs the following steps:

1. **Intent Routing:** The API detects terms such as “change” or “expiration” and routes the request to the **Planner Agent**.
2. **Planning Phase:** The Planner scans the codebase, retrieves the OTP verification logic in `auth.py`, and drafts a plan:

```
[
  {
    "action": "edit",
    "path": "backend/app/api/auth.py",
    "description":
      "Change expiration duration from
       minutes=15 to minutes=30"
  }
]
```

3. **Human Approval:** The change plan is rendered on the frontend. At this stage, no files are edited.

4. **Automation Phase:** After the user clicks “**Approve**”, the backend uses the `editing_tools.py` file editor to automatically make the required changes in the project files.
5. **Validation & Commit:** The user reviews the code changes (`git diff`) in the browser. If the changes are correct, they approve the commit, and RepoMind AI automatically saves the changes to Git with a commit.

2 Automated Test Suite Breakdown

RepoMind AI has a test suite located in `backend/tests/` that runs both unit and integration tests.

Current Test Modules

- **test_auth.py:** Verifies password hashing (`hash_password`), password checking (`verify_password`), and JWT session token creation and decoding.
- **test_commit_push_auth.py:** Tests the Git credential system, verifying that GitHub write-access tokens are validated properly.
- **test_file_intent_parser.py:** Checks the NLP intent parser to ensure that the system correctly extracts file paths and actions (such as *edit*, *create*, and *delete*) from user chat messages.
- **test_project_access.py:** Ensures that users can only access projects they own.
- **test_project_status.py:** Tests the state transitions of a project (e.g., *cloning*, *indexing*, *ready*, and *failed*).

3 Test Results & Execution Log

Here is the honest output of the test run executed on the backend:

```
===== test session starts =====
platform win32 -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\USER\OneDrive\Documents\New project 2\backend
configfile: pyproject.toml
plugins: anyio-4.13.0, asyncio-1.4.0
asyncio: mode=Mode.STRICT, debug=False
collected 34 items

tests\test_auth.py ... [ 8%]
tests\test_commit_push_auth.py ... [ 17%]
tests\test_file_intent_parser.py ..... [ 58%]
tests\test_language_instruction.py .. [ 64%]
tests\test_multilingual_investigation.py ... [ 73%]
tests\test_project_access.py .... [ 85%]
tests\test_project_status.py ..... [100%]
```

===== 34 passed in 5.92s =====

4 Assessment of Testing Gaps

To keep the system transparent, the following areas are not covered by the current automated test suite:

1. **Frontend Testing:**

There are no automated tests for the Next.js user interface. The frontend features are currently verified through manual testing. [3]

2. **No Live LLM Integration Tests:**

During testing, the system uses mock responses instead of making real API calls to Gemini, Groq, or Ollama. This approach saves execution time and API costs; however, real AI-generated responses are not automatically tested. [4]

3. **No Direct Database Integration Tests:**

The test suite uses mock implementations of MongoDB and ChromaDB rather than live database instances. Consequently, real database connectivity and vector indexing issues are not validated during automated testing. [9]

5 Manual Verification Flow (E2E)

Until End-to-End (E2E) test scripts are written, developers should perform the following manual checklist before shipping updates: [12]

1. **Ingestion Check:**

Import a test repository and ensure that its state changes from *cloning* → *indexing* → *ready*.

2. **Router Check:**

Type "Show project structure" and confirm that the user interface automatically redirects to the **Architecture** tab.

3. **Safety Edit Check:**

Type

`"Create a file test.txt with hello"`

and verify that the file is **not** created on disk until you click “**Approve Edit**” in the chat feed.

4. **Diff Validation:**

Confirm that the generated code diff correctly highlights additions in green and deletions in red.

Chapter 5

Real-World Comparison of RepoMind AI with Existing Research Papers, Projects & Tools

Real-World Problem	Existing Systems	RepoMind AI Solution	Result
A developer joins a large project containing hundreds of files and does not know where to start.	Most tools provide code suggestions or answer questions based on limited context. Understanding the repository often requires manual exploration.	RepoMind AI indexes the repository, creates vector embeddings, and understands the complete codebase before answering questions.	Developers can understand the project much faster and reduce onboarding time.
Finding the function responsible for a feature or bug is time-consuming.	Existing tools may search keywords but may miss semantically related code.	RepoMind AI performs semantic search using RAG and retrieves the most relevant code chunks.	Relevant files and functions are located quickly, saving development time.
Understanding the architecture of an unfamiliar repository is difficult.	Most assistants explain only the currently visible code or selected files.	RepoMind AI analyzes the repository and explains modules, components, and their relationships.	Developers gain a better understanding of the project structure.
Generating project documentation requires significant manual effort.	Documentation generation is limited or not repository-aware in many tools.	RepoMind AI generates documentation based on the actual repository content.	Documentation becomes more accurate and easier to create.
A bug may involve multiple files across the repository.	Many assistants focus on the current file and may not analyze dependencies across files.	RepoMind AI retrieves related files and provides repository-aware bug investigation.	Faster debugging and improved problem analysis.

Real-World Problem	Existing Systems	RepoMind AI Solution	Result
Developers often modify code without fully understanding its impact.	Existing tools may directly suggest edits without structured planning.	RepoMind AI first generates a change plan before any modification.	Users can review the proposed changes and reduce the risk of errors.
Automatic code modification may introduce unintended changes.	Some systems can edit code directly with limited user control.	RepoMind AI requires explicit human approval before applying changes.	Improves safety and keeps the developer in control.
Reviewing changes across multiple files can be difficult.	Diff visualization and structured review are limited in several systems.	RepoMind AI provides diff review before committing changes.	Easier verification and higher confidence in modifications.
Writing meaningful Git commit messages is repetitive and time-consuming.	Commit message generation is limited or absent in many tools.	RepoMind AI automatically generates commit messages based on the applied changes.	Saves time and improves version control documentation.
Developers need to switch between multiple tools for search, documentation, planning, review, and editing.	Existing solutions often specialize in one or two tasks instead of providing a unified workflow.	RepoMind AI integrates repository understanding, semantic search, planning, documentation, review, and commit assistance into a single system.	A unified development workflow increases productivity and reduces context switching.
Evaluating coding agents on software engineering tasks requires standardized benchmarks.	SWE-bench is designed as a benchmark rather than an interactive coding assistant.	RepoMind AI focuses on assisting developers during real repository analysis and code modification.	Provides practical development assistance instead of evaluation alone.
Organizations may prefer local AI models due to privacy concerns.	Some commercial assistants rely mainly on cloud services.	RepoMind AI supports both local models (e.g., Ollama) and cloud LLMs.	Greater flexibility for privacy-sensitive and offline environments.

Table : Comparison between Repomind AI and Other Projects[15][1], Research Papers

Overall Result

Existing AI coding assistants are generally strong in code completion or isolated development tasks, while RepoMind AI is designed to understand an entire repository and assist throughout the software development lifecycle. By combining Retrieval-Augmented Generation (RAG), semantic search, repository analysis, documentation generation, change planning, human approval, code review, and commit assistance, RepoMind AI provides a more integrated and transparent solution for modern software development.

Chapter 6

1 Limitations

Although RepoMind AI provides an effective repository-aware development assistant, the current implementation has several limitations that can be addressed in future versions to improve functionality, scalability, and reliability.

1. Works Only with Imported Repositories

RepoMind AI currently operates only on repositories that have been imported into the system. It does not yet support direct analysis of local project folders without the import process.

2. Re-indexing Required After Repository Changes

Whenever significant changes are made to the repository, the indexing process must be executed again to update the knowledge base and ensure accurate retrieval of repository information.

3. Limited Automated Testing

Frontend testing and live integration testing with Large Language Models (LLMs) are not fully automated. Certain testing and validation tasks still require manual verification.

4. Performance on Very Large Repositories

As repository size increases, indexing, retrieval, and analysis operations may require additional processing time, which can impact overall system responsiveness.

5. Dependency on Embedding and LLM Quality

The quality of generated answers, architecture summaries, documentation, and recommendations depends on the effectiveness of the embedding model and the selected Large Language Model.

6. Architecture Analysis May Require Validation

Although the system automatically generates architecture summaries and diagrams, complex repositories may still require developer review to verify the accuracy of the generated architectural representation.

7. Limited Support for Multiple Programming Languages

Repository understanding and analysis capabilities may vary across different programming languages and frameworks depending on available indexing and retrieval support.

8. Internet and API Dependency

Some AI-powered functionalities rely on external API services and internet connectivity. Temporary network or service interruptions may affect system availability and performance.

2 Future Scope

Although RepoMind AI successfully provides repository-aware code understanding and assistance, there are several areas where the system can be improved in the future.

1. **Support for Local Projects:** At present, RepoMind AI works only with imported repository trees. In the future, it can be extended to directly analyze and work with local project folders on a developer's computer without requiring repository import.
2. **Real-Time Repository Monitoring:** The current system indexes the repository only during import. Future versions can automatically detect file changes and update the vector database in real time without manual re-indexing.
3. **Support for More Programming Languages:** The current implementation focuses on common programming languages and project structures. Future versions can provide better support for additional programming languages and frameworks.
4. **Advanced Multi-Agent Collaboration:** More specialized AI agents can be added for tasks such as security analysis, performance optimization, code refactoring, and test case generation, enabling the system to solve more complex software engineering problems.
5. **Automated Frontend and Live Integration Testing:** The present testing framework mainly covers backend functionality using mock components. Future development can include automated frontend testing and live integration testing with Large Language Models (LLMs) and databases.
6. **IDE Integration:** RepoMind AI can be integrated directly with development environments such as Visual Studio Code and IntelliJ IDEA, allowing developers to use its features without leaving their coding environment.
7. **Support for Team Collaboration:** Future versions can include multi-user collaboration, shared repository knowledge, role-based access control, and collaborative code review for software development teams.
8. **Improved Code Modification and Refactoring:** The system can be enhanced to automatically perform complex multi-file refactoring while maintaining human approval and safety checks before applying changes.

9. **Cloud-Based Repository Management:** Future versions can support direct integration with platforms such as GitHub, GitLab, and Bitbucket, enabling automatic synchronization and repository analysis.
10. **Smarter AI Reasoning:** By using more advanced Large Language Models (LLMs) and improved Retrieval-Augmented Generation (RAG) techniques, RepoMind AI can provide more accurate explanations, better planning, and higher-quality code suggestions for large and complex repositories.

3 Conclusion

RepoMind AI is a repository-aware AI coding assistant that helps developers understand and work with software projects more easily. Instead of only generating code, it studies the entire repository, searches for relevant information, and provides answers based on the actual project content.

The system combines repository indexing, semantic search, documentation generation, bug investigation, change planning, and approval-based editing into a single workflow. By requiring user approval before applying changes, it ensures that code modifications are safe and transparent.

The project also demonstrates the practical use of modern technologies such as Retrieval-Augmented Generation (RAG), vector databases, embeddings, and Large Language Models (LLMs). The testing results show that the implemented features work correctly and provide a reliable foundation for repository analysis and code assistance.

Although there is scope for future improvements, such as automated frontend testing and live LLM integration testing, RepoMind AI successfully achieves its main objective of making repository understanding and software development faster, easier, and safer. It can serve as a useful tool for students, developers, and software teams working on large and complex codebases.

References

- [1] Zhang, Q., Chen, J., Valamehr, R., & Wang, S. (2023). *RepoCoder: Repository-Level Code Completion*. arXiv preprint arXiv:2303.12570.
- [2] Jimenez, C. E., Yang, J., Wetzler, A., & Narasimhan, K. (2023). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* arXiv preprint arXiv:2310.06770.
- [3] Nussbaum, Z., Cooley, J., & Kirkpatrick, J. (2024). *Nomic Embed: Training a Reusable Text Embedding*. arXiv preprint arXiv:2402.01613.
- [4] Yang, J., Jimenez, C. E., Wetzler, A., & Narasimhan, K. (2024). *SWE-agent: Agent-Computer Interfaces Enable LLMs to Resolve Real-World GitHub Issues*. arXiv preprint arXiv:2405.15793.
- [5] Lewis, P., Perez, E., Piktus, A., Petroni, F., & Yih, W. T. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459–9474.
- [6] Liu, T., Xu, C., & Zhang, Y. (2023). *RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems*. arXiv preprint arXiv:2306.03091.
- [7] Dong, Y., Jiang, X., & Zhang, J. (2023). *AgentCoder: Multi-Agent-Based Code Generation with Iterative Testing and Self-Correction*. arXiv preprint arXiv:2312.13010.
- [8] Hong, S., Zheng, L., Chen, J., & Wang, L. (2023). *MetaGPT: Meta Programming for Multi-Agent Collaborative Framework*. arXiv preprint arXiv:2308.00352.
- [9] Qian, C., Cong, F., & Yang, M. (2023). *Communicative Agents for Software Development*. arXiv preprint arXiv:2309.07864.
- [10] Chen, M., Tworek, J., Jun, H., & Zaremba, W. (2021). *Evaluating Large Language Models Trained on Code*. arXiv preprint arXiv:2107.03374.
- [11] Chen, X., Wang, M., & Song, D. (2023). *Self-Debugging: Teaching Language Models to Self-Debug Code*. arXiv preprint arXiv:2304.05128.
- [12] Rozière, B., Gehring, J., Gloeckle, F., & Lample, G. (2023). *Code Llama: Open Foundation Models for Code*. arXiv preprint arXiv:2308.12950.
- [13] Leopard AI Team. (2024). *RepoQA: Evaluating Long-Context Needle-in-a-Haystack Retrieval in Codebases*. Technical Report.
- [14] Brunsfeld, M., et al. (2023). *Tree-sitter: An Incremental Parsing Library for Programming Tools*. Tree-sitter Project Documentation.

- [15] Landais, M., & GitPython Contributors. (2024). *GitPython: A Python Library Used to Interact with Git Repositories*. GitPython Documentation.